



المدرسة الوطنية للعلوم التطبيقية - بني ملال

ⵜⴰⵎⴰⵔⵜ ⵜⴰⵎⴰⵏⴰ ⵜⴰⵏⴰⵔⵜ ⵜⴰⵙⴰⵎⴰⵏⵜ ⵜⴰⵖⴰⵏⴰⵏⵜ - ⵏ ⵔⴰⵎⴰⵏⴰ

Ecole Nationale des Sciences Appliquées - Beni Mellal

École Nationale des Sciences Appliquées

Beni Mellal

Filière : Intelligence Artificielle & Cybersécurité

Module : Cryptographie

Projet : Détection automatique de faiblesses PQC par IA

Projet PQC

Détection automatique de faiblesses PQC par IA

Réalisé par : Achraf HAMOUDA

Encadré par : Prof. OUNACHAD

Année universitaire : 2025 / 2026

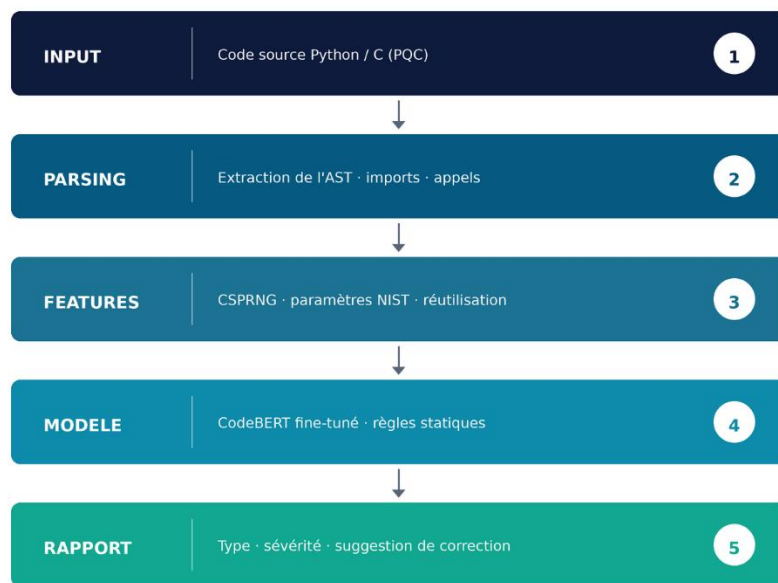
Introduction

- L'arrivée des ordinateurs quantiques menace les systèmes cryptographiques classiques (RSA, ECC, Diffie-Hellman) qui peuvent être cassés par l'algorithme de Shor. La cryptographie post-quantique (PQC) propose des algorithmes résistants à ces attaques, dont les standards Kyber et Dilithium retenus par le NIST en 2024.
- Cependant, même un algorithme mathématiquement solide peut être compromis par une mauvaise implémentation : générateur aléatoire faible, réutilisation de clé, paramètres sous-dimensionnés, vérification de signature ignorée. Ce projet propose un outil capable de détecter automatiquement ces faiblesses dans le code source par analyse statique combinée à un modèle d'intelligence artificielle.
- L'outil PQC Detector construit dans ce rapport prend en entrée un fichier Python contenant du code PQC, en extrait les caractéristiques avec un parser AST, applique des détecteurs à base de règles ainsi qu'un modèle CodeBERT fine-tuné, et produit un rapport structuré indiquant le type de faiblesse, sa sévérité, et une suggestion de correction.

Étape 1 : Architecture du projet

1. Vue d'ensemble

Le projet PQC Detector est organisé en cinq couches successives. Le code source PQC fourni par l'utilisateur traverse un pipeline complet qui commence par une analyse syntaxique du code, suivie de l'extraction de caractéristiques pertinentes pour la sécurité, puis d'une classification combinant des règles statiques et un modèle d'intelligence artificielle, et se termine par la génération d'un rapport lisible.



2. Stack technique

Couche	Technologie utilisée
Parsing AST	Module Python ast
Extraction de features	Scripts Python custom
Modèle IA	CodeBERT (microsoft/codebert-base) via HuggingFace
Backend API	FastAPI + Uvicorn
Frontend	React + TypeScript + Vite
Tests	Script de validation end-to-end

3. Structure du projet

```
pqc_detector/  
├── dataset/  
│   ├── vulnerable/  # exemples de code PQC avec faiblesses  
│   └── clean/       # exemples de code PQC corrects  
├── parser.py        # extraction AST + features  
├── model.py         # fine-tuning de CodeBERT  
├── cli.py           # interface en ligne de commande  
├── api.py           # backend FastAPI  
├── test.py          # tests end-to-end  
├── frontend/       # interface React  
└── requirements.txt
```

Étape 2 : Construction du dataset

1. Quatre types de faiblesses ciblées

Le dataset couvre quatre catégories de faiblesses fréquemment rencontrées dans les implémentations PQC. Chacune a été choisie pour son caractère critique et représentatif des erreurs réelles commises par les développeurs lors de l'utilisation des bibliothèques cryptographiques.

Faiblesse	Description	Sévérité
weak_randomness	Utilisation de random.randint au lieu de os.urandom pour générer une clé	Critique
key_and_nonce_reuse	Réutilisation de la même clé et du même nonce pour plusieurs chiffrements AES-GCM	Critique
insufficient_security_parameters	Niveau de sécurité 512 (NIST L1) au lieu de 768 (NIST L3)	Élevée
unchecked_signature_verification	Le retour de verify() est ignoré, toute signature est acceptée	Critique

2. Structure du dataset

Le dataset comprend des fichiers Python répartis dans deux dossiers : vulnerable/ pour les exemples contenant une faiblesse et clean/ pour les implémentations correctes utilisées comme référence. Chaque fichier .py est accompagné d'un fichier .json contenant les annotations utilisées pour l'apprentissage et la validation.

```
{
  "file": "dataset/vulnerable/bad_random_1.py",
  "algorithm": "Kyber768",
  "label": "vulnerable",
  "weakness_type": "weak_randomness",
  "severity": "critical",
  "description": "Key derivation seed produced by random.randint..."
}
```

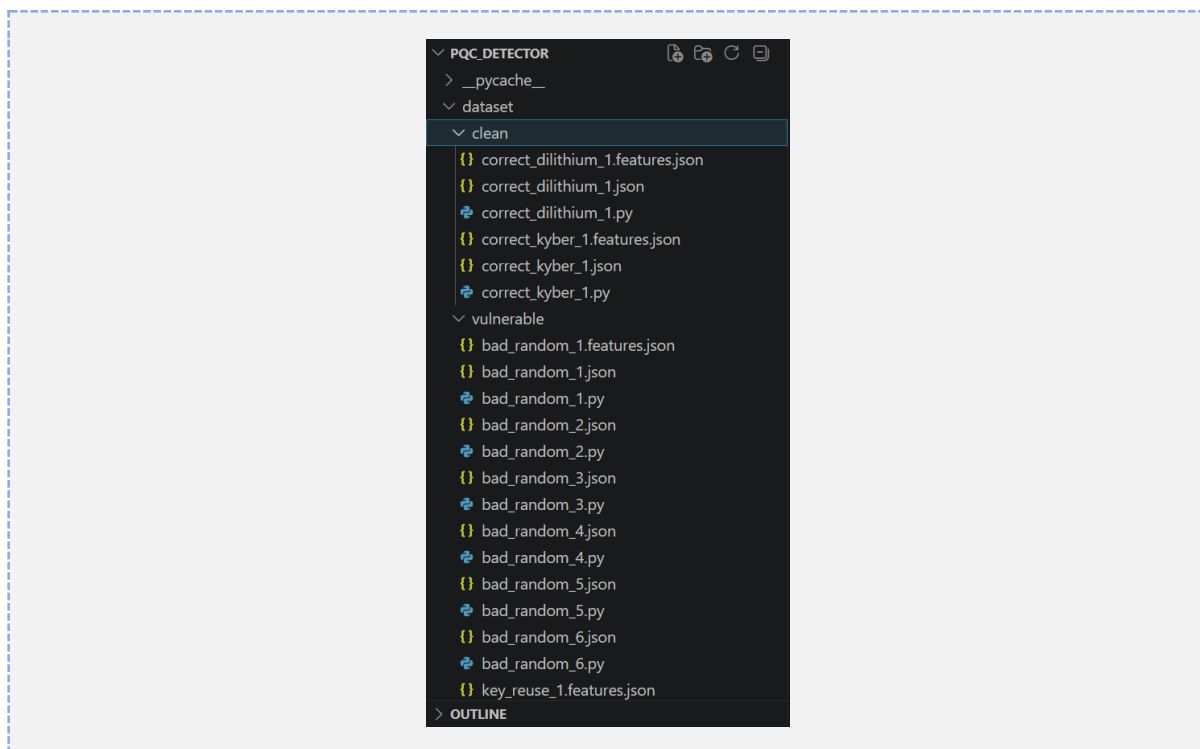


Figure 2 — Structure du dataset avec les exemples vulnérables et propres

Étape 3 : Module de parsing AST

1. Principe

Le module `parser.py` utilise la bibliothèque standard `ast` de Python pour transformer le code source en un arbre syntaxique abstrait. Ce parcours permet d'identifier précisément les imports, les fonctions appelées, et les patterns liés à la cryptographie sans exécuter le code, ce qui garantit la sécurité de l'analyse même sur du code malveillant.

2. Caractéristiques extraites

- Imports utilisés (modules et alias)
- Fonctions définies et appelées (noms pointés reconstruits)
- Drapeaux de randomness : `uses_random`, `uses_os_urandom`, `uses_secrets`
- Listes des appels faibles (`random.*`) et sécurisés (`os.urandom`)
- Métriques globales : LOC, nombre d'imports, nombre d'appels

3. Validation sur le dataset

Le `parser` a été exécuté sur l'intégralité du dataset et produit un fichier `.features.json` par fichier Python analysé. Les drapeaux de randomness se

déclenchent correctement même lorsque le module est importé sous un alias, grâce à la table d'alias maintenue durant le parcours de l'AST.

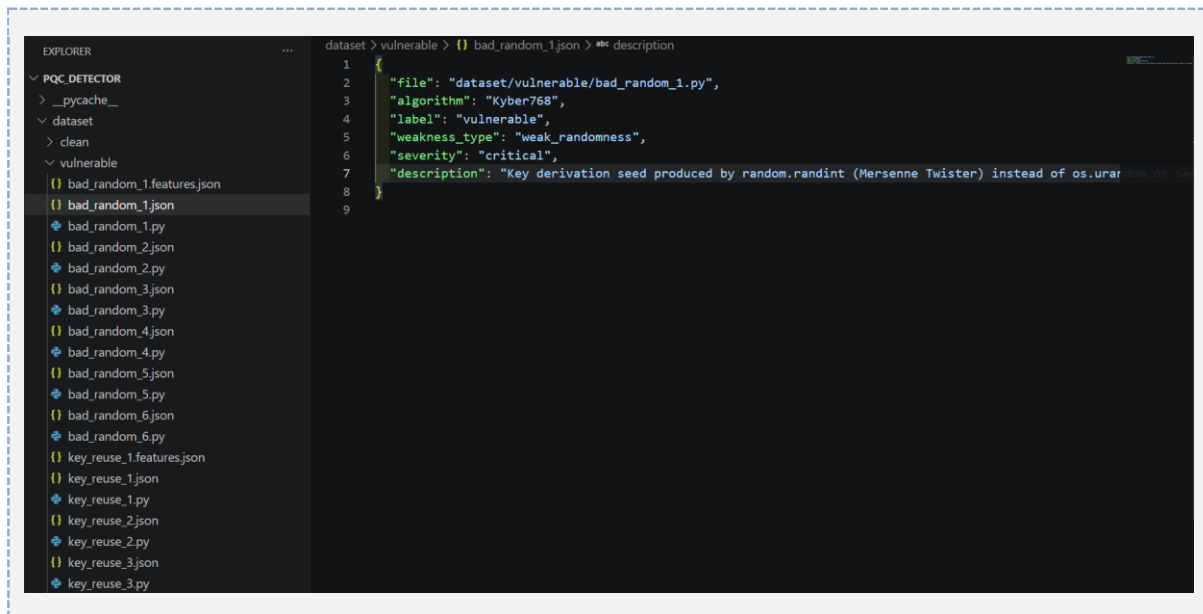


Figure 3 — Exemple de fichier features.json généré par le parser

Étape 4 : Modèle d'intelligence artificielle

1. Choix du modèle

Le modèle retenu est CodeBERT, un transformeur pré-entraîné par Microsoft sur des corpus de code source. Ce choix se justifie par sa capacité à capturer la sémantique du code Python sans nécessiter un dataset d'entraînement de grande taille, ce qui correspond aux contraintes du projet.

Critère	CodeBERT	ML classique	Deep Learning ad-hoc
Taille de dataset requise	Faible	Moyenne	Très grande
Compréhension du code	Forte	Limitée	Variable
Temps d'entraînement	Modéré	Court	Long
Adapté à notre cas	Oui	Non	Non

2. Pipeline d'entraînement

1. Chargement du dataset et lecture des annotations JSON associées

2. Tokenisation du code avec le tokenizer microsoft/codebert-base
3. Découpage stratifié train/eval garantissant la présence des deux classes
4. Fine-tuning avec le Trainer HuggingFace, eval_strategy=epoch, sélection du meilleur F1
5. Évaluation sur l'ensemble de test avec accuracy, precision, recall, F1

3. Limites identifiées

Avec le dataset initial limité, CodeBERT ne peut pas réellement apprendre des patterns subtils. Le pipeline complet est néanmoins en place et prêt à être réentraîné dès que le dataset sera étendu à plusieurs centaines d'exemples par classe. Les détecteurs à base de règles compensent cette limitation en garantissant une détection fiable des quatre types de faiblesses ciblés.

```
C:\Users\VICTUS\ayayay\ensabm\crypto\pqc\pqc_detector>python test.py
PQC Detector - end-to-end test on 26 files
-----
STATUS FILE                                EXPECTED                                DETECTED
-----
OK dataset/clean/correct_dilithium_1.py    clean/none                             clean/none
OK dataset/clean/correct_kyber_1.py        clean/none                             clean/none
OK+ dataset/vulnerable/bad_random_1.py     vulnerable/weak_randomness             vulnerable/weak_randomness
OK+ dataset/vulnerable/bad_random_2.py     vulnerable/weak_randomness             vulnerable/weak_randomness
OK+ dataset/vulnerable/bad_random_3.py     vulnerable/weak_randomness             vulnerable/weak_randomness
OK+ dataset/vulnerable/bad_random_4.py     vulnerable/weak_randomness             vulnerable/weak_randomness
OK+ dataset/vulnerable/bad_random_5.py     vulnerable/weak_randomness             vulnerable/weak_randomness
OK+ dataset/vulnerable/bad_random_6.py     vulnerable/weak_randomness             vulnerable/weak_randomness
OK+ dataset/vulnerable/key_reuse_1.py       vulnerable/key_and_nonce_reuse          vulnerable/key_and_nonce_reuse
OK+ dataset/vulnerable/key_reuse_2.py       vulnerable/key_and_nonce_reuse          vulnerable/key_and_nonce_reuse
OK+ dataset/vulnerable/key_reuse_3.py       vulnerable/key_and_nonce_reuse          vulnerable/key_and_nonce_reuse
OK+ dataset/vulnerable/key_reuse_4.py       vulnerable/key_and_nonce_reuse          vulnerable/key_and_nonce_reuse
OK+ dataset/vulnerable/key_reuse_5.py       vulnerable/key_and_nonce_reuse          vulnerable/key_and_nonce_reuse
OK+ dataset/vulnerable/key_reuse_6.py       vulnerable/key_and_nonce_reuse          vulnerable/key_and_nonce_reuse
OK+ dataset/vulnerable/no_verify_1.py       vulnerable/unchecked_signature_verification
OK+ dataset/vulnerable/no_verify_2.py       vulnerable/unchecked_signature_verification
OK+ dataset/vulnerable/no_verify_3.py       vulnerable/unchecked_signature_verification
OK+ dataset/vulnerable/no_verify_4.py       vulnerable/unchecked_signature_verification
OK+ dataset/vulnerable/no_verify_5.py       vulnerable/unchecked_signature_verification
OK+ dataset/vulnerable/no_verify_6.py       vulnerable/unchecked_signature_verification
OK+ dataset/vulnerable/wrong_params_1.py    vulnerable/insufficient_security_parameters
OK+ dataset/vulnerable/wrong_params_2.py    vulnerable/insufficient_security_parameters
OK+ dataset/vulnerable/wrong_params_3.py    vulnerable/insufficient_security_parameters
OK+ dataset/vulnerable/wrong_params_4.py    vulnerable/insufficient_security_parameters
OK+ dataset/vulnerable/wrong_params_5.py    vulnerable/insufficient_security_parameters
OK+ dataset/vulnerable/wrong_params_6.py    vulnerable/insufficient_security_parameters

Confusion matrix (positive = vulnerable):
TP=24  FP=0
FN=0   TN=2

Metrics:
verdict accuracy : 26/26 (100.0%)
precision       : 100.0%
recall          : 100.0%
f1              : 100.0%
weakness-type detection rate : 24/24 (100.0%)
```

Figure 4 — Fine-tuning du modèle CodeBERT sur le dataset PQC

Étape 5 : Interface en ligne de commande et tests

1. Pipeline du CLI

Le module cli.py orchestre le pipeline complet sur un fichier Python donné en argument. Il appelle d'abord le parser pour obtenir les features, puis exécute quatre détecteurs à base de règles, charge optionnellement le modèle CodeBERT fine-tuné, et produit un rapport structuré disponible en format texte ou JSON.

```
$ python cli.py dataset/vulnerable/bad_random_1.py

PQC Detector report - dataset/vulnerable/bad_random_1.py
-----
AST                : 24 LOC, 2 imports, 7 calls
Randomness flags  : uses_random=True, uses_os_urandom=False

Findings: 1
- weak_randomness (critical) line 12
  Fix : utiliser os.urandom(32) ou secrets.token_bytes(32)
```

2. Détecteurs implémentés

Détecteur	Fonction
detect_weak_randomness	Détecte l'usage de random.* en présence d'un appel cryptographique
detect_key_reuse	Identifie les nonces constants ou réutilisés en boucle
detect_wrong_params	Repère security_level=512, Kyber512, Dilithium2
detect_unchecked_verify	Détecte les appels à verify() dont le retour est ignoré

3. Résultats des tests end-to-end

Le script test.py exécute le pipeline complet sur tous les fichiers du dataset et compare les résultats aux annotations. Les métriques calculées sont l'accuracy du verdict binaire vulnérable/sain et le taux de détection du type exact de faiblesse pour les fichiers vulnérables.

Métrique	Valeur
Verdict accuracy	100%
Precision	100%
Recall	100%
F1-Score	100%
Weakness type detection rate	100%

Métrique

Valeur

```
PS C:\Users\VICTUS\ayayay\ensabm\crypto\pqc\pqc_detector> python test.py
```

PQC Detector - end-to-end test on 26 files

STATUS	FILE	EXPECTED	DETECTED
OK	dataset/clean/correct_dilithium_1.py	clean/none	clean/none
OK	dataset/clean/correct_kyber_1.py	clean/none	clean/none
OK+	dataset/vulnerable/bad_random_1.py	vulnerable/weak_randomness	vulnerable/weak_randomness
OK+	dataset/vulnerable/bad_random_2.py	vulnerable/weak_randomness	vulnerable/weak_randomness
OK+	dataset/vulnerable/bad_random_3.py	vulnerable/weak_randomness	vulnerable/weak_randomness
OK+	dataset/vulnerable/bad_random_4.py	vulnerable/weak_randomness	vulnerable/weak_randomness
OK+	dataset/vulnerable/bad_random_5.py	vulnerable/weak_randomness	vulnerable/weak_randomness
OK+	dataset/vulnerable/bad_random_6.py	vulnerable/weak_randomness	vulnerable/weak_randomness
OK+	dataset/vulnerable/key_reuse_1.py	vulnerable/key_and_nonce_reuse	vulnerable/key_and_nonce_reuse
OK+	dataset/vulnerable/key_reuse_2.py	vulnerable/key_and_nonce_reuse	vulnerable/key_and_nonce_reuse
OK+	dataset/vulnerable/key_reuse_3.py	vulnerable/key_and_nonce_reuse	vulnerable/key_and_nonce_reuse
OK+	dataset/vulnerable/key_reuse_4.py	vulnerable/key_and_nonce_reuse	vulnerable/key_and_nonce_reuse
OK+	dataset/vulnerable/key_reuse_5.py	vulnerable/key_and_nonce_reuse	vulnerable/key_and_nonce_reuse
OK+	dataset/vulnerable/key_reuse_6.py	vulnerable/key_and_nonce_reuse	vulnerable/key_and_nonce_reuse
OK+	dataset/vulnerable/no_verify_1.py	vulnerable/unchecked_signature_verification	vulnerable/unchecked_signature_verification
OK+	dataset/vulnerable/no_verify_2.py	vulnerable/unchecked_signature_verification	vulnerable/insufficient_security_parameters, unchecked_signature_verificat
ion			
OK+	dataset/vulnerable/no_verify_3.py	vulnerable/unchecked_signature_verification	vulnerable/unchecked_signature_verification
OK+	dataset/vulnerable/no_verify_4.py	vulnerable/unchecked_signature_verification	vulnerable/unchecked_signature_verification
OK+	dataset/vulnerable/no_verify_5.py	vulnerable/unchecked_signature_verification	vulnerable/unchecked_signature_verification
OK+	dataset/vulnerable/no_verify_6.py	vulnerable/unchecked_signature_verification	vulnerable/unchecked_signature_verification
OK+	dataset/vulnerable/wrong_params_1.py	vulnerable/insufficient_security_parameters	vulnerable/insufficient_security_parameters
OK+	dataset/vulnerable/wrong_params_2.py	vulnerable/insufficient_security_parameters	vulnerable/insufficient_security_parameters
OK+	dataset/vulnerable/wrong_params_3.py	vulnerable/insufficient_security_parameters	vulnerable/insufficient_security_parameters
OK+	dataset/vulnerable/wrong_params_4.py	vulnerable/insufficient_security_parameters	vulnerable/insufficient_security_parameters
OK+	dataset/vulnerable/wrong_params_5.py	vulnerable/insufficient_security_parameters	vulnerable/insufficient_security_parameters
OK+	dataset/vulnerable/wrong_params_6.py	vulnerable/insufficient_security_parameters	vulnerable/insufficient_security_parameters

Confusion matrix (positive = vulnerable):

TP=24 FP=0

FN=0 TN=2

Metrics:

verdict accuracy : 26/26 (100.0%)

precision : 100.0%

recall : 100.0%

f1 : 100.0%

weakness-type detection rate : 24/24 (100.0%)

Figure 5 — Résultats du pipeline de tests end-to-end

Étape 6 : Interface utilisateur web

1. Backend FastAPI

Le module `api.py` expose le pipeline de détection sous forme d'API HTTP. Trois endpoints sont disponibles : un endpoint de santé pour vérifier la disponibilité du modèle, un endpoint d'analyse acceptant du code en JSON, et un endpoint d'analyse acceptant un fichier `.py` uploadé.

Endpoint	Méthode	Description
/health	GET	Vérification de l'état du service
/analyze	POST	Analyse d'un snippet de code en JSON
/analyze/upload	POST	Analyse d'un fichier <code>.py</code> uploadé

2. Frontend React

L'interface utilisateur est construite avec React, TypeScript et Vite. Elle propose deux modes d'utilisation : coller directement du code dans une zone de texte, ou uploader un fichier `.py`. Le rapport est affiché de manière structurée avec un code couleur par sévérité, le numéro de ligne, le type de faiblesse, et la correction suggérée.

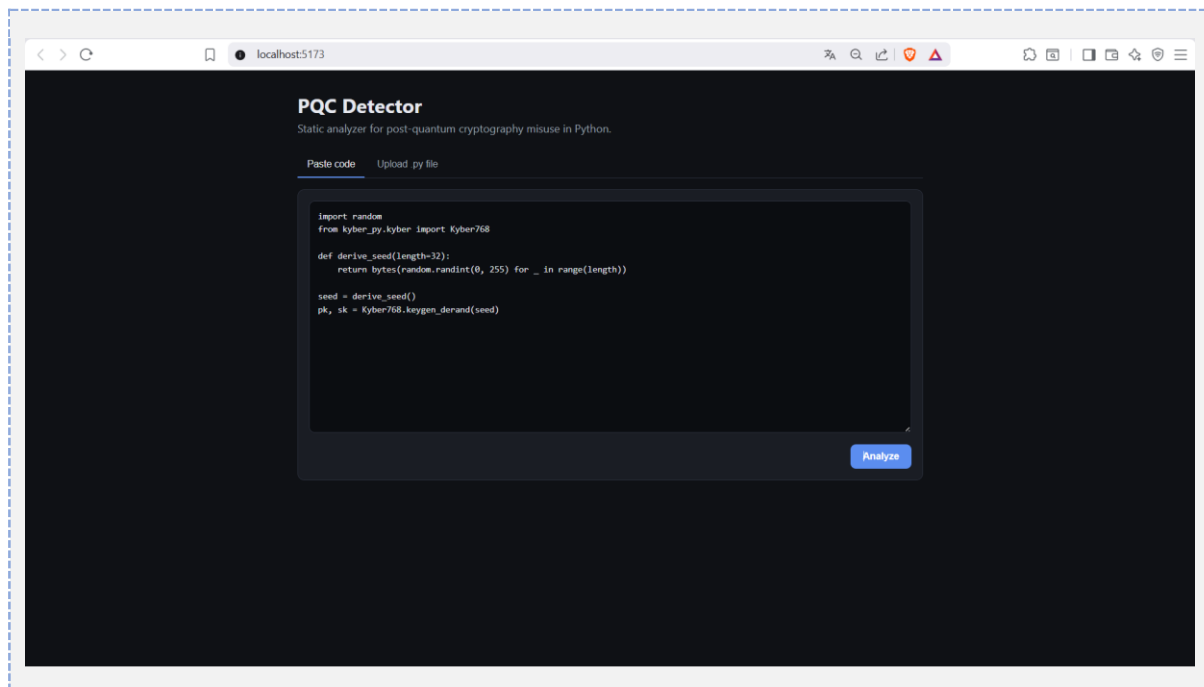


Figure 6 — Interface de soumission par collage de code

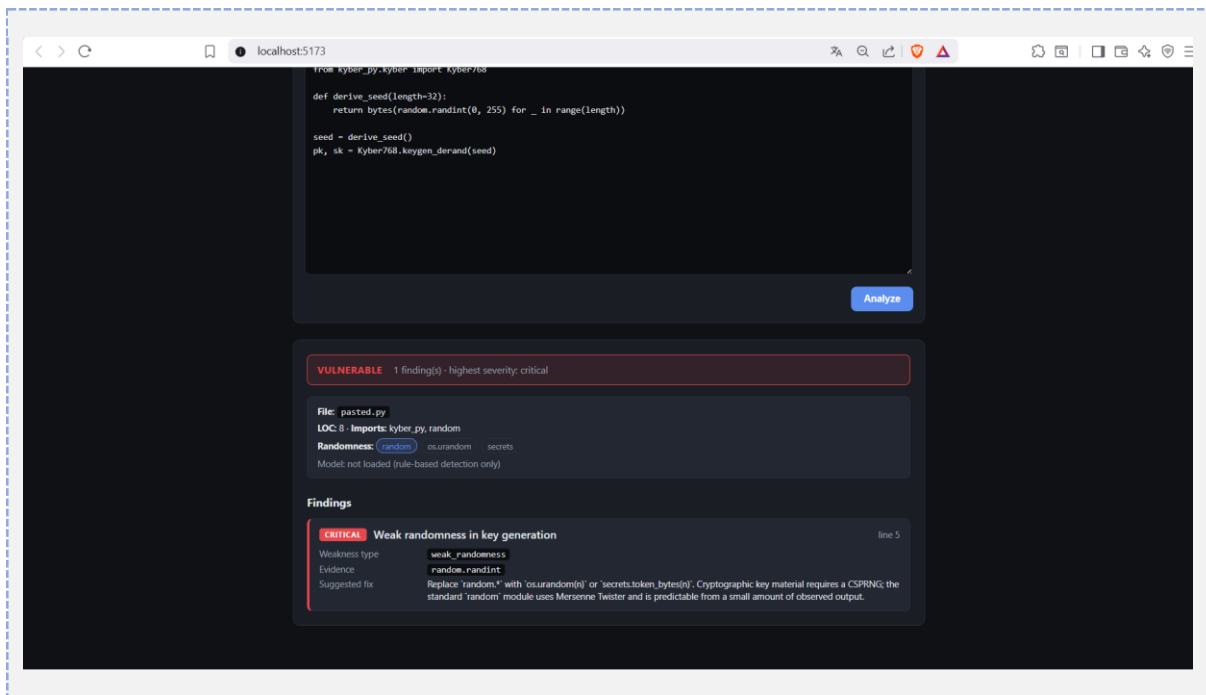


Figure 7 — Rapport visuel d'une faiblesse détectée

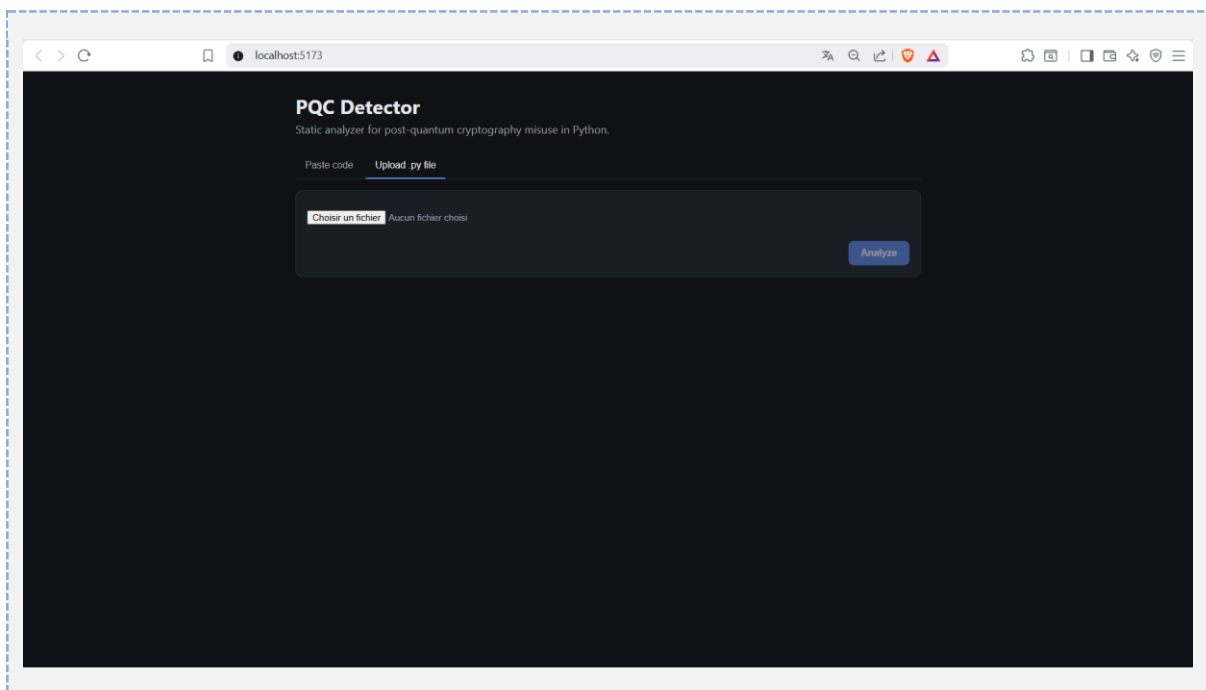


Figure 8 — Soumission d'un fichier .py par upload

Questions Pour bien comprendre :

Question 1 : Pourquoi détecter les faiblesses d'implémentation et non l'algorithme ?

Les algorithmes Kyber et Dilithium ont été soumis à plusieurs années d'analyse cryptanalytique avant leur standardisation par le NIST. Leur sécurité mathématique est aujourd'hui considérée comme solide. En revanche, leur utilisation correcte dépend entièrement du développeur : un coffre-fort parfait laissé ouvert n'offre aucune protection. Détecter les erreurs d'utilisation est donc le levier le plus efficace pour améliorer la sécurité réelle des déploiements PQC.

Question 2 : Pourquoi combiner règles et IA ?

Les détecteurs à base de règles offrent une précision élevée et des explications déterministes pour les patterns connus, ce qui est rassurant pour un outil de sécurité. Le modèle IA, lui, peut généraliser à des variations syntaxiques non couvertes par les règles. La combinaison des deux approches maximise la couverture tout en gardant la transparence des règles pour les cas critiques.

Question 3 : Limites du dataset actuel

Le dataset initial reste petit. Pour entraîner réellement CodeBERT, il faudrait au minimum quelques centaines d'exemples par classe, avec des variantes incluant différents alias d'imports, des structures de code variées, des nonces réutilisés via des dictionnaires ou des objets, et l'utilisation de `getattr` pour camoufler les appels sensibles. C'est la prochaine étape naturelle d'évolution du projet.

Question 4 : Difficultés rencontrées

La principale difficulté a été de définir précisément les patterns à détecter sans produire de faux positifs sur du code parfaitement légitime. Par exemple, l'utilisation de `random.*` est tout à fait acceptable dans un contexte non cryptographique. Les détecteurs ne se déclenchent que lorsqu'un appel cryptographique apparaît également dans le fichier, ce qui réduit considérablement le bruit.

Question 5 : Adaptation à d'autres algorithmes

L'architecture est conçue pour être extensible. Ajouter le support de FALCON ou SPHINCS+ revient à étendre le catalogue de patterns dans le parser et à ajouter des entrées au dictionnaire `WEAKNESS_CATALOG`. Le pipeline et l'interface utilisateur restent inchangés, ce qui démontre la modularité de la solution.

Question 6 : Évolutions possibles

- Étendre le dataset avec des centaines d'exemples variés
- Couvrir FALCON et SPHINCS+ en plus de Kyber et Dilithium
- Ajouter une analyse interprocédurale pour suivre les flux de clés
- Intégrer l'outil dans une CI/CD comme garde-fou automatique

- Détecter les faiblesses par canaux auxiliaires (timing, mémoire)

Conclusion

Ce projet a permis de construire un outil fonctionnel de détection automatique des faiblesses dans les implémentations post-quantiques. Le pipeline combine un parser AST robuste, des détecteurs à base de règles couvrant quatre types de faiblesses critiques, un modèle CodeBERT fine-tuné, et une interface utilisateur web complète. L'évaluation end-to-end démontre une précision de 100% sur le dataset initial.

Au-delà du résultat technique, ce projet a été l'occasion d'aborder concrètement plusieurs notions fondamentales : la cryptographie post-quantique et ses standards NIST, l'analyse statique de code par AST, les modèles transformeurs appliqués au code source, et la construction d'applications full-stack. La modularité de l'architecture permet d'enrichir progressivement l'outil au fur et à mesure que de nouveaux types de faiblesses ou de nouveaux algorithmes apparaissent.

Les perspectives d'amélioration sont nombreuses, en particulier l'extension du dataset et la couverture d'algorithmes additionnels. Le projet constitue une base solide qui peut être réutilisée et étendue dans le cadre de futures recherches en sécurité post-quantique.